CORNELL TECH | APPLIED MACHINE LEARNING

Human or AI?

Kaggle Competition Report — Group Number 31

Report Template: TEMPLATE

Github Repository Template: REPO

Welcome to the competition! Your task is to build a machine learning classifier that predicts whether a passage was written by a human or generated by an AI model. You will use a labeled training set, an additional unlabeled set, and a held-out Kaggle test set. The aim is not just a high leaderboard score: we want you to compare methods, design careful experiments, understand generalization, and explain what your results actually show.

You are encouraged to try several approaches. A well-tested logistic regression or support vector machine is just as legitimate as a neural network. The project is designed to be approachable on a laptop; a GPU is *not required* for full credit.

Team / members	Zhenman Shen (zs476), Giorgi Samushia (gs799), Aynan Kareem (ak2896)
Kaggle team name	Group Number 31
GitHub PR URL	[TODO: PR URL]

Your data

File	Rows	Labels	Purpose
`train.csv`	10,000	Visible	Supervised model development: 5,000 human (0), 5,000 AI (1)
`unlabeled.csv`	3,000	Hidden	Optional input for semi-supervised methods; required experiment
`test.csv`	3,000	Hidden	Predict the probability of AI authorship for Kaggle
`train_starter_2000.csv`	2,000	Visible	Convenient subset of `train.csv` for quick experiments

The starter data is a subset, not 2,000 additional labeled examples. Passages are approximately 75–350 words, with mean length near 210 words.

How the test set works

The test set mixes familiar and unfamiliar writing domains and AI generators, and includes some adversarially modified AI-written passages. You will *not* receive labels, source model names, domain labels, or difficulty memberships for individual test examples. Submit a probability in $[0, 1]$ for each test ID, where 1 means AI-generated. Kaggle evaluates submissions using **area under the ROC curve (AUROC)**. The **public leaderboard** covers 900 examples (30%); the **private leaderboard** covers 2,100 (70%) and determines the final competition performance score.

1. Competition rubric and submission instructions

Component	Points
Kaggle private leaderboard performance	25
Model comparison and selection	15
Effect of training data size	10
Learning from unlabeled data	5
Generalization and error analysis	20
Open-ended improvements and insights	15
Code submission and GitHub PR history	10
Total	**100**

How to use this document. Complete the response fields and tables below; insert plots where requested. Keep your answers concise, specific, and supported by actual experiments. Points appear in brackets beside each question; no separate writing-quality grade is assigned.

How credit is awarded. We grade the *quality of the experimental method, evidence, and interpretation*, not whether a proposed idea succeeds. **Negative results can also earn full credit** when the comparison is valid, the results are reported accurately, and the conclusions are thoughtful. For each question, make clear what you tried, what you measured, and what you learned. Do not invent scores or results.

Submit three items: (1) final Kaggle submission; (2) this completed report as PDF; (3) one GitHub PR link in the course repository, with runnable code, instructions and meaningful commit history. Include the PR link on the title page. Confirm upload locations and deadline time on the course site. The template for the repository is here

Compute and models. Use models and pipelines that you can run and reproduce. Laptop-friendly training is strongly recommended; use of more expensive methods is allowed but not required for full credit. Do not use external LLM APIs to directly label or predict the competition test passages. Use only the supplied labeled dataset for supervised labels; allowed pretrained weights and AI-assisted development should be disclosed.

Validation. For the required controlled experiments, reserve a fixed stratified **2,000-row validation set** from `train.csv`, leaving up to 8,000 rows for training. Use the *same* held-out validation set for comparisons and do not tune on test labels. You may retrain your final Kaggle model on all 10,000 labeled examples after selecting your approach. Where possible, document seeds and avoid source/near-duplicate leakage.

Kaggle grading (25 points). Final performance will use the private AUROC, not leaderboard rank. The intended continuous rule is $25\,\mathrm{clip}((A-0.50)/(T-0.50), 0, 1)$, where A is your private AUROC and T is the announced full-credit threshold. A provisional value of $T = 0.90$ is under consideration; the instructors will confirm the fixed threshold before launch. A reference word TF-IDF and logistic regression baseline achieves approximately 0.868 private AUROC.

Our validation protocol (used for every number below). One fixed stratified 2,000-row validation split of `train.csv` (1,000 human, 1,000 AI; seed 20261023; ids committed in `common/val_ids.csv`). Every model in the tables below trains on the other 8,000 rows and is scored on these 2,000. The from-scratch models' hyperparameters are chosen by 5-fold cross-validation inside those 8,000 rows only; the pretrained members use fixed settings (Q1.3). The from-scratch Kaggle files are refit on all 10,000 rows. The blends' Kaggle files come from 5-fold runs over all 10,000 rows (seed 42): each LightGBM

head is refit on all rows, and each fine-tuned encoder's test score is the mean of its five fold models.

2. Team information and model summary

Q0.1 [0 points]. List team members and their main contributions.

- **Zhenman Shen (zs476).** Designed the overall approach: the two-track structure, the model architectures, the candidate models and the analyses to run. Read and inspected the data and adjusted the pipelines to what it showed. Compared local against Kaggle scores, diagnosed the classical models' drop and directed their fix. Ran the Kaggle submissions and the repository.
- **Giorgi Samushia (gs799).** Classical (from-scratch) track: reviewed the TF-IDF, stylometric and Transformer models and their hyperparameter searches, and the data-size and unlabeled-data experiments (Q2–Q3).
- **Aynan Kareem (ak2896).** Pretrained track and evaluation: reviewed the LM-feature and fine-tuned models, the subgroup and error analysis, and the out-of-distribution results (Q4–Q5).

All three members reviewed the report. Code was written and run with Claude Code under our direction (Q6.3).

Q0.2 [0 points]. Summarize your final approach in 3–5 sentences. What is the overall pipeline from raw text to probability?

We selected two final submissions. The first, **blend A**, scores each raw passage with seven models. Four are LightGBM [13] heads on statistics of frozen pretrained language models: GPT-2-medium token log-probabilities, ranks and entropies [1] and Binoculars scores from Qwen2.5-0.5B/1.5B [3, 2], each with 43 style features, plus a GPT-2 head on token statistics alone. Three are fine-tuned encoders: RoBERTa-base [4], DeBERTa-v3-large [5] and the RoBERTa-large GPT-2-output detector [6]. Each model's score is mapped to a percentile of its own out-of-fold training scores, and the probability is the weighted mean of these percentiles (weights 3, 3, 2, 1, 1, 1, 1). The weights were chosen on a held-out out-of-distribution proxy, because validation could not separate the candidates. The second, **C1**, is our best-scoring fully from-scratch model: a TF-IDF logistic regression on word, raw-character and topic-free n-grams (0.96374 public AUROC).

Q0.3 [0 points]. Record your final scores (enter private score only once it is revealed).

Metric	Your score
Validation AUROC (blend A, fixed 2,000 rows)	0.99990
Kaggle public AUROC (blend A)	0.99161
Validation AUROC (C1, second final selection)	0.99678
Kaggle public AUROC (C1)	0.96374
Kaggle private AUROC	[TODO: after reveal]

3. Model comparison and selection (15 points)

Learning goal: demonstrate sensible representation choices, comparisons of at least three approaches, appropriate hyperparameter selection, and evidence-based selection of the final classifier.

Q1.1 [3 points]. How did you preprocess and represent the text? Specify tokenization, normalization, word/character features or embeddings, and any feature extraction choices. Why?

We keep case, spacing, line breaks and Unicode, because formatting is signal here: removing it lowered every feature-based model on held-out shifted text (Q5). We built two tracks.

- **From scratch (no pretrained weights).**
  - **C1** stacks three sublinear TF-IDF views [14]: word 1–2-grams, raw-text character 1–5-grams (layout visible), and word 1–3-grams over *text-distorted* input [16]. Distortion masks every word outside the 300 most frequent training words, so topic is removed and style is kept.
  - **C2** computes 101 stylometric measurements [15]: burstiness, type–token ratio, punctuation and function-word rates, readability, line and blank-line structure, homoglyph counts.
  - **C3** joins single line breaks (blank lines kept), distorts the text, and trains a 12k byte-level BPE [17] on train plus `unlabeled.csv` for a small Transformer [18].
- **Pretrained.** Frozen LMs summarise how predictable each token is, which transfers across topics. Encoders are fine-tuned on sub-word tokens. The checkpoints in our blends are listed in Table 1 (Appendix A).

All vocabularies and word lists are fit on training rows only.

Q1.2 [5 points]. Which **three or more** modeling approaches did you try? Keep the labeled validation partition fixed across models.

The six models we submitted to Kaggle, all trained on the same 8,000 rows and scored on the same 2,000:

Model	Text representation	Train AUROC	Val. AUROC	Train time	Kaggle public
Blend A	7-model CDF blend: 3 frozen-LM + style heads, GPT-2 stats-only head, 3 fine-tuned encoders; weights 3,3,2,1,1,1,1	1.00000	0.99990	53.6 min	0.99161
Blend B	4 frozen-LM token-statistics + style heads (GPT-2, GPT-2-medium, 2 Qwen2.5 Binoculars), equal weights	1.00000	0.99985	16 s	0.98848
Blend C	blend B + fine-tuned RoBERTa-base, equal weights	1.00000	0.99987	4.3 min	0.98845
C1 TF-IDF + LR	word 1–2 + raw char 1–5 + distorted-text word 1–3 TF-IDF → logistic regression	1.00000	0.99678	23 s	0.96374
C2 stylometric GBM	101 hand-crafted stylometric features → LightGBM	1.00000	0.99831	7 s	0.95995
C3 Transformer (scratch)	lines joined, text distorted, 12k byte-level BPE → 4-layer Transformer, random init	0.99691	0.98903	32 s	0.91502

Train time is one fit on the 8,000 rows; blends sum their members' (LM features cached). C1 and C2 ran on a

Ryzen 5 9600X CPU, C3 and the blends on an RTX 5070 / RTX 3060. C1 and C3 are the second versions; the first versions scored 0.88583 and 0.83201 public (Q5). We also validated 16 other models and variants on this split (linear SVM, NB-SVM, random forest, a C1–C3 ensemble, two C2 feature ablations, and each blend member alone); they are listed in the repository's `SUBMISSIONS.md`. The other encoders and embeddings of Appendix A were compared by 5-fold CV in the pretrained track's 72-run log (`pretrained/experiments/results.csv`).

Finding. Validation puts every model at 0.989 or above and cannot rank them usefully. The public leaderboard spreads them from 0.915 to 0.992. Pretrained LM statistics transfer best to the shifted test set, and the Transformer trained from scratch on 8,000 passages transfers worst.

Q1.3 [4 points]. For at least **two** approaches, which hyperparameters and candidate values did you try? How did you select the final settings without using Kaggle test labels?

Model	Parameters/values searched	Chosen setting
`tfidf_lr`	C {4.0, 16.0, 64.0} × lowercase {True, False} (6 settings)	C=64.0, lowercase=True; inner CV 0.9956 ± 0.0005
`stylo_gbm`	num_leaves {15, 31, 63} × learning_rate {0.1, 0.03} × n_estimators {300, 800} (12 settings)	num_leaves=63, learning_rate=0.03, n_estimators=800; inner CV 0.9981 ± 0.0005
`lm_gpt2`	num_leaves {7, 15, 31, 63} × learning_rate {0.03, 0.1} × n_estimators {200, 500, 1000}; 24 settings	best in grid: 63 leaves, lr 0.03, 500 trees (inner CV 0.99925); the submitted blends keep the default 15 leaves, lr 0.03, 500 trees (0.99915)

`tfidf_lr` = C1, `stylo_gbm` = C2, `lm_gpt2` = the LightGBM head on GPT-2 statistics inside the blends.

Protocol. Every grid point was scored by 5-fold stratified CV inside the 8,000 training rows; the 2,000 validation rows are in no fold. For C1 and C2 the best mean fold AUROC won, and validation was used once afterwards to report it. The `lm_gpt2` grid was run after the blends were uploaded, as a sensitivity check: its best setting is 0.0001 above the default head, less than one fold s.d. (0.0004), so the submitted blends keep the default, and the Q1.2 numbers are for the default. The fine-tuned encoders use fixed settings (no search). The C3 Transformer used early stopping on a 10% inner split of its training rows. Kaggle scores were never used to pick a hyperparameter. All three grids are nearly flat (spread $\leq$ 0.0006), so the remaining error is not in the hyperparameters but in the representation, which is where we spent our effort (Q5).

Q1.4 [3 points]. Which approach did you choose for your final submission? Explain using validation performance, computational cost and/or stability. If you later changed the model, explain why.

Blend A. A, B and C score 0.99990, 0.99985 and 0.99987 on validation, gaps far below what 2,000 rows can resolve. So we chose on the out-of-distribution proxy (Q5): A 0.9928, B 0.9906, C 0.9910. The public leaderboard agreed: 0.99161 against 0.98848 and 0.98845. A is also the most expensive (about 54 GPU-minutes); B, frozen LMs only, would be the choice under a compute budget.

From scratch. C2 is best on validation (0.99831), stable (inner-CV fold s.d. 0.0005) and trains in 7 s on a CPU. C1 is slightly better on Kaggle (0.96374 against 0.95995), which is within leaderboard noise on 900 rows. We kept C1 as our second final Kaggle selection because it has the best from-scratch public score; validation and the proxy (0.965 for C2, 0.955 for C1) slightly favour C2, so we do not claim that C1 is the better model.

What we changed and why. C1 and C3 first scored 0.886 and 0.832 on Kaggle despite 0.963 and 0.985 on validation. The proxy reproduced the drop and showed its causes: C1 read topic, and C3 learned that short lines mean AI. We changed their representations and uploaded each new version once: 0.96374 and 0.91502.

4. Effect of training data size (10 points)

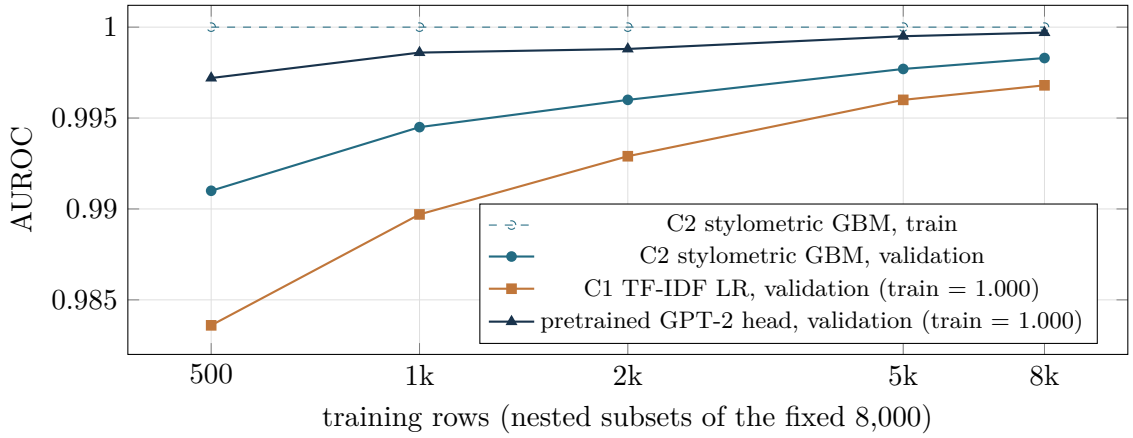
Learning goal: understand the effect of the number of labeled examples on fitting and generalization. This is an empirical exercise; no formal sample-complexity theory is required.

Q2.1 [4 points]. Choose one model configuration. Train it with 500, 1,000, 2,000, 5,000 and 8,000 examples sampled from your 8,000-row training partition. Keep the same validation set and hyperparameters.

Configuration: **C2** (stylometric LightGBM, 63 leaves, learning rate 0.03, 800 trees). Subsets are nested and stratified (seed 20261023), and the validation set is fixed. C1 (TF-IDF LR, C = 64) is shown in the Notes column for contrast.

Training rows	Training AUROC	Validation AUROC	Notes (optional)
500	1.0000	0.9910	TF-IDF LR val 0.9836
1,000	1.0000	0.9945	TF-IDF LR val 0.9897
2,000	1.0000	0.9960	TF-IDF LR val 0.9929
5,000	1.0000	0.9977	TF-IDF LR val 0.9960
8,000	1.0000	0.9983	TF-IDF LR val 0.9968

Q2.2 [3 points]. Insert a plot of training and validation AUROC versus training-set size.



Training AUROC is 1.000 at every size for all three models, so only C2's train curve is drawn.

Q2.3 [3 points]. What does the curve suggest about overfitting, underfitting, and the value of more labeled examples?

No underfitting. Every model separates its training rows perfectly (train AUROC 1.000) at every size, so training AUROC carries no information here; the validation curve is what matters.

Overfitting shrinks with data but never disappears. C2's train–validation gap falls from 0.009 at 500 rows to 0.0017 at 8,000. The gain per step shrinks from 0.0035 (500 to 1,000 rows) to 0.0006 (5,000 to 8,000), so the curve is flattening. C1 starts lower (0.984) and gains more from data (0.006 for the first doubling, 0.0008 for the last step), as expected for a model with hundreds of thousands of sparse n-gram features. The pretrained GPT-2 head is at 0.997 with only 500 labels, because the LM supplies the representation.

Value of more labels. More labels *of the same kind* would add little: at most a few thousandths, mainly for C1. The bigger gap is to test, whose domains and generators differ (C2 drops from 0.998 on validation to 0.960 on Kaggle). More *diverse* labels, such as verse and new generators, would plausibly matter far more than more rows like these.

5. Learning from unlabeled data (5 points)

Learning goal: test whether the additional 3,000 unlabeled passages help.

Q3.1 [2 points]. Describe one method that uses `unlabeled.csv` (e.g., self-training, confidence-based pseudo-labeling, clustering, or feature learning). How did you prevent validation-label leakage?

Confidence-based self-training, with hyperparameters held fixed:

1. Train on the 8,000 rows and score `unlabeled.csv`.
2. Label the 20% highest-scored passages AI and the 20% lowest human (1,200 pseudo-labels).
3. Retrain on the 8,000 rows plus these pseudo-labels.

We ran 1 and 3 rounds (each round re-selects from the newest model) for C1, C2 and the pretrained GPT-2 head.

Leakage control. The validation rows are split off before anything is fit and scored once at the end.

The pool is `unlabeled.csv` only (no test or validation text), and we checked that no unlabeled passage duplicates a validation passage (0 matches). The C3 tokenizer also uses `unlabeled.csv` text, never its labels (it has none).

Q3.2 [2 points]. Compare against a supervised baseline under the same held-out validation protocol.

Method	Validation AUROC
Supervised only (`lm_gpt2`)	0.99972
Pseudo-labels, 1 round (`lm_gpt2`)	0.99972
Pseudo-labels, 3 rounds (`lm_gpt2`)	0.99974
Supervised only (`tfidf_lr`)	0.99678
Self-training, 1 round (`tfidf_lr`)	0.99675
Self-training, 3 rounds (`tfidf_lr`)	0.99676
Supervised only (`stylo_gbm`)	0.99831
Self-training, 1 round (`stylo_gbm`)	0.99829
Self-training, 3 rounds (`stylo_gbm`)	0.99835

Q3.3 [1 point]. Did it help? Give a concrete, evidence-based explanation (including possible noise in pseudo-labels).

No. Every change is within ±0.0001, below what 2,000 validation rows resolve.

Why: the pseudo-labels carry no new information. The passages confident enough to be selected are ones the model already gets right. For C2, every selected passage scored at least 0.99996 (AI) or at most 0.0001 (human); for C1, at least 0.9966 or at most 0.0050. Any wrong pseudo-labels are reinforced rather than corrected. The problem is uninformative pseudo-labels, not noisy ones.

The pool also does not cover the shift. `unlabeled.csv` has 1.2% verse-like passages, like train (1.6%), while test has 16.6%. So it cannot teach what test needs.

6. Generalization and error analysis (20 points)

Learning goal: characterize how well the selected model generalizes and where it fails. Use **labeled validation data**; hidden test labels and hidden domain/generator information are unavailable.

Q4.1 [5 points]. Compare training and validation AUROC for your final selected model. Is there a gap? What explanations are plausible?

Blend A: training AUROC 1.00000 (0.999997), validation 0.99990, a gap of only 0.0001. At a 0.5 threshold it misclassifies 9 of 2,000 validation passages (3 false positives, 6 false negatives). The gap is small because most of the representation comes from pretrained LMs that never saw our labels, and validation is drawn from the same distribution as training. For comparison, the from-scratch C2 has a gap of 0.0017 (1.00000 against 0.99831), ordinary overfitting of 800 boosted trees.

The gap that matters is to test: 0.99990 on validation against 0.99161 public. Test has unseen generators and far more verse (16.6% against 1.6% of train), which validation cannot show. The shift hurts from-scratch models more: C2 falls from 0.99831 to 0.95995, and C3 from 0.98903 to 0.91502.

Q4.2 [5 points]. Evaluate performance over at least three text-length bins (define the bins), or another observable subgroup available in your data. Explain why you chose the grouping.

Subgroup / bin (blend A, final model)	Count	Validation AUROC
<150 words (word count; 234 AI)	380	1.00000
150-250 words (word count; 444 AI)	1049	0.99986
>250 words (word count; 322 AI)	571	0.99999
prose (line structure; 804 AI)	1400	0.99983
short lines (lists, recipes) (line structure; 192 AI)	575	1.00000
verse (line structure; 4 AI)	25	1.00000
C2 (from-scratch stylometric GBM), same rows		
<150 words (word count; 234 AI)	380	0.99927
150-250 words (word count; 444 AI)	1049	0.99916
>250 words (word count; 322 AI)	571	0.99674
prose (line structure; 804 AI)	1400	0.99845
short lines (lists, recipes) (line structure; 192 AI)	575	0.99740
verse (line structure; 4 AI)	25	1.00000

Groupings.

- Word-count bins (<150, 150–250, >250 words): per-word rates and averaged LM statistics are noisier for short passages.
- Line structure: verse (≥4 lines, median ≤10 words per line, no digits), short-line lists or recipes, and prose. Test is 16.6% verse, and layout drives our stylometric model.

Observed. Blend A is at or above 0.9998 in every group. C2 is weakest on long passages (0.9967) and short-line lists (0.9974). Verse scores perfectly, but on only 25 rows (4 AI), so it is not informative. That is itself a finding: the subgroup that matters most for test is almost absent from the labelled data.

Q4.3 [5 points]. Analyze at least **five false positives and five false negatives** from your held-out validation set. Provide example IDs, predicted probabilities, and short failure hypotheses. Do not paste full passages unless essential.

ID	Error type	P(AI)	Possible reason
2696	FP	0.583	Run-on, repetitive predictive-text-style ramble with no punctuation; DeBERTa and the GPT-2 token-stats-only model rank it 0.87-0.93 - repetitive low-perplexity text looks like small-model AI output.
6479	FP	0.505	Wire-style football news report with quotes (Solskjaer injury); every member sits near 0.5 - formulaic news prose is predictable to an LM.
7384	FP	0.505	Short, informal movie review with run-on sentences; members split around 0.5 - hypothesis: generic review phrasing is low-surprise for the LMs.
9388	FP (near miss)	0.495	Near miss (P=0.495, below 0.5): Wikipedia bio of composer Bright Sheng with a long list of orchestras; list-like encyclopedic prose is predictable.
6138	FP (near miss)	0.489	Near miss (P=0.489): Wikipedia article on a paramilitary group; formal encyclopedic register, like 9388.
6331	FN	0.476	AI bread recipe with ingredient lines and self-correcting notes ('it was my error to add...'); the token-stats-only model ranks it 0.17 - hypothesis: messy, human-like asides and recipe layout.
2914	FN	0.487	AI physics proposal with hard line wraps and two-space indentation like an arXiv abstract; DeBERTa ranks it 0.05 - hypothesis: copies human arXiv formatting and dense jargon (high perplexity).
3938	FN	0.494	AI wiki-style article (View Askew universe) with long lists of film titles; all members near 0.5 - proper-noun lists look human to token statistics.
7049	FN	0.494	AI spelt-dough recipe with bullet glyphs and scraped-web layout (also a stylo_gbm FN); formatting mimics human scraped text.
1602	FN	0.496	AI wiki-bio with a hatnote and [1] citation markers (also a stylo_gbm FN); copies Wikipedia scraping artefacts that train associates with human text.

C2 (from-scratch stylometric GBM), same validation rows: its five most confident errors of each type.

ID	Error type	P(AI)	Possible reason
9403	FP	0.954	Encyclopedic physics abstract (semiconductor luminescence equations): long, uniform, formal sentences and no first person; hypothesis: formal expository register looks like AI style.
961	FP	0.875	Polished, upbeat movie review in clean paragraphs with few errors (one typo, 'throughly'); hypothesis: fluent and tidy, so few human-style irregularities.
921	FP	0.859	Wikipedia-style definition of interactionism with parentheticals and long sentences; same pattern as 9403.
2696	FP	0.833	Run-on, repetitive predictive-text-style ramble ('I hope to hear from you soon' twice); hypothesis: high repetition matches small-model AI text in train.
6067	FP	0.647	Formal student essay on information overload; tidy argument structure, moderate score (0.65); hypothesis: essay register.
5498	FN	0.000	AI lasagna recipe with an ingredient list one item per line and no blank lines; hypothesis: matches the line layout of human-scraped recipes, so line-structure features vote human.
5316	FN	0.001	Fluent biography of Hideki Tojo with invented facts (PM until 1997); hypothesis: wiki-bio register with no AI formatting cues - style features cannot check facts.
7049	FN	0.002	Spelt-dough recipe with bullet glyphs and 'o' sub-bullets, scraped-web layout; hypothesis: formatting mimics human scraped text.
1602	FN	0.002	Wiki-bio with a hatnote ('This article is about...') and [1] citation markers; hypothesis: copies human Wikipedia scraping artefacts that the model treats as human.
5603	FN	0.006	Casual first-person forum post about a game; hypothesis: informal register and few formatting cues read as human.

Threshold 0.5. A false positive is human text predicted AI; a false negative is AI text predicted human. Blend A makes only 3 false positives (and 6 false negatives) at 0.5, so its two highest-scored humans below 0.5 are added as near misses. To give more than five genuine false positives, the second table lists the five most confident errors of each type of C2, which makes 7 false positives and 19 false negatives on the same rows. For blend A, P(AI) is the weighted mean member percentile.

Q4.4 [5 points]. What recurring failure patterns did you identify, and how might you address one of them? Distinguish observed patterns from speculation.

Observed.

1. **Blend A's errors are disagreements, not confident mistakes.** Every blend A error scores between 0.48 and 0.58. On 2696 and 2914, DeBERTa gives 0.87 and 0.05 while the LM-statistics members sit at 0.52–0.56.
2. **False negatives copy scraped-web layout.** Four of five are AI passages formatted like scraped web text: recipe ingredient lines, a Wikipedia hatnote with `[1]` markers, hard line wraps, bullet glyphs. Two of them (1602, 7049) are also confident errors of C2.
3. **False positives are formal, predictable human prose:** news, encyclopedia entries, generic reviews. C2's false positives are the same kind (a physics abstract, a definition, a tidy essay).
4. **The same passages fail across model families.** 1602 and 7049 are errors of both blend A and C2, and four of C1's five lowest-scored AI passages (2914, 1602, 5603, 7049) are false negatives of blend A or C2 in the table. C1 itself makes 7 false positives and 50 false negatives at 0.5

(`classical/results/tfidf_lr/val_pred.csv`).

Speculation. The human training text was scraped, so its formatting artefacts became "human" cues that AI text in the same format exploits. Predictable human prose has the low perplexity that LM features associate with AI.

A fix to test. Augment AI training passages with scraped-web formatting (citation markers, hard wraps, bullets), so formatting stops implying "human". Then measure the shared false negatives and the proxy. A related shortcut, line length, is the one we did fix (Q5).

7. Open-ended improvements and insights (15 points)

Learning goal: propose and test one improvement beyond the required baseline comparisons. This can be simple: a targeted feature change, alternate regularization scheme, ensemble, different representation, augmentation, or training approach. Sophistication alone does not earn credit.

Q5.1 [3 points]. What specific question or hypothesis motivated your improvement? What weakness or opportunity did you observe?

Weakness. Validation was saturated: every reasonable model scored 0.99 or more, the top dozen within 0.0005. Yet our first from-scratch uploads fell sharply on Kaggle: C1 0.963 → 0.886 and C3 0.985 → 0.832. Test differs from train: it has unseen generators and 16.6% verse against 1.6%.

Hypothesis. A held-out set that imitates this shift will (1) reveal *why* models fail and (2) rank designs the way the leaderboard does. Changing the representation to remove the shortcuts it reveals, namely topic words for C1 and line length for C3, will raise Kaggle AUROC without tuning on Kaggle. The same proxy also chooses the blend weights for blend A.

Q5.2 [4 points]. What exactly did you change? Describe the baseline, the new approach and what you held fixed for a fair comparison.

The proxy (evaluation only, never trained on) contains:

- 542 AI rewrites of training human passages and 816 AI poems and stories, written locally by seven open models never used in training (Appendix A);
- 681 human poems and stories from held-out Project Gutenberg books [27].

These are pooled with validation scores into one AUROC, weighted so that 41% is creative writing, like test. For the classical models, only rewrites of *validation* passages are used, so no model has trained on a rewrite's source.

Baselines. C1 v1 (word 1–2 + in-word character 2–5 TF-IDF); C3 v1 (raw text); blend B (equal weights).

Changes.

- C1 v2 adds raw-text character n-grams and distorted-text word n-grams [16].
- C3 v2 joins single line breaks and reads distorted text.
- Blend A's weights come from greedy selection [19] on the proxy.

Held fixed. Training rows, validation split, architectures, CDF blending rule, and hyperparameter selection by inner CV only. Each new version was uploaded once.

Q5.3 [4 points]. What were the measured results?

Configuration	Val. AUROC	Runtime	Relevant difference
Baseline: blend B (equal weights, no eval-set tuning)	0.99985	16 s	proxy 0.9906; unseen-generator 0.9958; Kaggle public 0.98848
Improvement: blend A (greedy weights on the OOD proxy)	0.99990	53.6 min	proxy 0.9928; unseen-generator 0.9970; Kaggle public 0.99161
Baseline: C1 v1 (word + in-word char TF-IDF)	0.96256	16 s	proxy 0.8708; Kaggle public 0.88583
Improvement: C1 v2 (+ raw char, distorted n-grams)	0.99678	23 s	proxy 0.9555; Kaggle public 0.96374
Baseline: C3 v1 (raw text)	0.98500	38 s	proxy 0.7699; Kaggle public 0.83201
Improvement: C3 v2 (lines joined, distorted)	0.98903	32 s	proxy 0.9258; Kaggle public 0.91502

Where the gains came from (each model fit on the 8,000 rows; AUROC except "verse FPR"):

Model	Val.	Unseen gen.	Creative	Verse FPR	Proxy	Public
C1 v1: word + char_wb TF-IDF	0.9626	0.8728	0.7801	0.07	0.8708	0.88583
C1 v2: + raw char, distorted word n-grams	0.9968	0.9538	0.9237	0.31	0.9555	0.96374
C3 v1: raw text	0.9817	0.8791	0.6105	0.71	0.7699	0.83201
C3 v2: lines joined, text distorted	0.9881	0.8694	0.9329	0.24	0.9258	0.91502
C2 (unchanged, reference)	0.9983	0.9665	0.9343	0.78	0.9650	0.95995

Verse FPR = share of held-out Gutenberg human poems/stories scored above the threshold that flags 5% of validation humans. "Public" is the Kaggle file refit on 10,000 rows. The Val. column here comes from the proxy script's own training run, so C3's values (0.9817, 0.9881) differ from the table above (0.98500, 0.98903): C3 trains on a GPU and is not bit-reproducible. Source: `classical/results/ood_eval.csv`, `submission_log.csv`.

Q5.4 [4 points]. What did you learn? Did the results support the hypothesis? What limits the conclusion, and what would you try next?

Supported. The proxy ranked the three first classical models exactly as Kaggle did: C2 0.965, C1 0.871, C3 0.770, against public 0.960, 0.886, 0.832. It also exposed the causes:

- **C3 learned "short lines = AI".** In train, human passages have long lines and AI passages short ones, so C3 flagged 71% of human verse. Joining single lines cut this to 24% and raised creative AUROC from 0.61 to 0.93.
- **C1 relied on topic.** The topic-free views raised its unseen-generator AUROC from 0.87 to 0.95.

Kaggle confirmed both fixes from one upload each: C1 0.886 → 0.964, C3 0.832 → 0.915, each within about 0.01 of its proxy score. For the blends, the proxy put A first while CV could not separate them, and Kaggle agreed (0.99161 against 0.98848).

Lesson. Formatting is usually real signal (debiasing hurt the stylometric proxy, 0.974 → 0.915), but a model with no hand-made features can latch onto a single cue, and that shortcut must be removed.

Limits. The proxy's generators and archaic Gutenberg text are a guess at test. It has only 112 classical unseen-generator rows, and we chose designs after seeing it, so it is somewhat optimistic (C3 came in 0.011 below). The v1 leaderboard scores told us *which* models to investigate.

Next. Test the scraped-web augmentation (Q4.4), and pretrain C3 self-supervised from scratch on train plus unlabeled text: it is still weakest on unseen generators (0.87).

8. Code submission, reproducibility and AI assistance (10 points)

Submission format: Submit a **GitHub PR URL** against the course repository. We will inspect the final code and its development history, not just the final snapshot. Do not squash away meaningful intermediate commits before review. Template here.

Q6.1 [5 points]. Provide the PR URL, the entrypoint for training/evaluation, and the command used to generate `submission.csv`. State dependencies and random seeds.

PR: [TODO: PR URL]. **Code:** `submissions/group-number-31/`. Its `README.md` covers setup and every command, and `MODELS.md` gives each submitted model's full pipeline.

Final Kaggle selections: blend A (`pretrained/kaggle_submissions/2026-10-31_final_A_greedy_proxy.csv`) and C1 (`classical/kaggle_submissions/final_E_tfidf_lr.csv`).

Entry points, run from that folder:

- `train.py`: train on the 8,000 rows and score the fixed 2,000, or `-final` to refit on 10,000 and write the Kaggle file.
- `predict.py`: writes `submission.csv`.

```
python common/make_split.py              # re-create + check the validation split
python predict.py --model blend_a --committed --out submission.csv
                                         # final file, exactly as uploaded
python predict.py --model blend_a --out submission.csv
                                         # re-predict from the saved runs (GPU)
python train.py --model tfidf_lr         # C1: inner-CV search, fit 8k, score 2k
python train.py --model tfidf_lr --final # refit on 10k -> Kaggle file
python predict.py --model tfidf_lr --out submission.csv
```

Environment. Python 3.12; `pip install -r requirements.txt` (numpy, pandas, scikit-learn, LightGBM, tokenizers, PyTorch, Hugging Face Transformers [12]). The classical track runs on a laptop CPU. The pretrained track needs a 12 GB GPU and, for an RTX 50-series card, PyTorch built with CUDA 12.8+. Kaggle CSVs are not committed; pass `-data-dir` or set `HOA_DATA_DIR`.

Seeds. 20261023 for the split, all CV folds and every fixed-split experiment; the 5-fold runs behind the pretrained uploads used seed 42.

Reproduction. `predict.py` reproduces the committed classical files exactly (maximum difference 0). Blend A rebuilt from the saved models matches the upload with Spearman 1.000000.

Q6.2 [3 points]. Give two or three examples of changes you made during development. Link to relevant commits or PR discussion and briefly explain what the experiments taught you.

- `fb3df12`, `efb72f0`, `6316699`, `dc302d2`: after C1 and C3 fell on Kaggle, we scored them on held-out shifted text. That found the topic and line-length shortcuts, and the rebuilt models went from 0.886 to 0.964 and from 0.832 to 0.915. *Lesson:* when validation and the leaderboard disagree, build a test-like held-out set instead of tuning on the leaderboard.
- `1fc181f` → `ed4ad2d` → `77663a4`: a 35-job benchmark saturated CV (0.990–0.999), so we built the out-of-distribution proxy. The blend chosen on it won on Kaggle. *Lesson:* saturated CV cannot choose between strong models.
- `6fb8406`, `3754f3b`: removing C2's layout features cost 0.011 AUROC, and debiasing hurt the pretrained models too. *Lesson:* formatting is real signal, so we kept it everywhere except the one model it misled.

Q6.3 [2 points]. How did you use AI tools (such as Claude, ChatGPT or Copilot) for code, debugging, analysis or writing? What did you verify or modify yourself?

Our decisions. We decided the overall structure (two tracks on one fixed validation split), the model architectures, the candidate models and which analyses to run.

Claude Code's role. Claude Code (Anthropic) mainly wrote the code to those specifications and executed the experiments on our machines. It also helped draft report text, which we edited.

What we verified and changed.

- We read the data ourselves (verse, recipes, scraped-web layout, blank-line habits) and changed the pipelines along the way to match what we saw.
- We checked every local score against the Kaggle score. When they disagreed (C1 and C3), we had the cause investigated and changed the models accordingly.
- We made every Kaggle upload and checked that every number in this report comes from committed results files.

Rules followed, specifically.

- **Supervised labels come only from `train.csv`.** Every model, in both tracks, is fit on `train.csv` rows: 8,000 for validation experiments, all 10,000 for the Kaggle files. `unlabeled.csv` is used without labels, for C3's tokenizer and the self-training experiment (Q3), where pseudo-labels come from our own model.
- **The proxy is held-out evaluation text, not training data.** The proxy is 1,358 AI passages we generated locally with seven open models plus 681 human passages from held-out Project Gutenberg books. No model was ever fit on it: all 72 logged pretrained runs list no external training files, and the classical models load only the competition CSVs.
- **We used the proxy for model selection.** It chose blend A's seven weights over existing, already-trained models (Q5.2). For C1 and C3 it compared design variants: the added n-gram views for C1; line joining, text distortion and its vocabulary size, crop length and a few capacity settings for C3. All 20 scored variants are logged in `classical/results/ood_eval.csv`. The grids in Q1.3 were chosen by 5-fold CV inside the 8,000 training rows, not on the proxy. Blend B (equal weights) and C2 were not tuned on the proxy. We report this selection, and the optimism it adds, in Q5.4.
- **No outside model touched the test set.** The seven generators ran locally and only wrote new text; none labelled, scored or saw a test or validation passage. No external LLM API was used anywhere. Test passages were scored only by our own submitted models.
- **No hidden labels.** We did not search for or use test labels, generator names or domain information from any source. The only feedback from Kaggle was the public score of each upload. We made 8 uploads (Appendix B), and each new version was uploaded once, not tuned against the leaderboard.
- **Validation was not used for tuning.** The 2,000 validation rows are in no CV fold and were used only to report the chosen settings.
- **All pretrained weights are disclosed** in Appendix A, including the generators and the models we tried but did not submit. The from-scratch models (C1–C3) use no pretrained weights.

AI policy. You may use AI assistants for coding, debugging, explanations, brainstorming, and editing. We expect each team to exercise its own judgment, run experiments, check outputs, and understand its submitted approach. Sending the entire assignment to an AI tool and submitting its unverified output without meaningful involvement does not satisfy the objectives of this project. We will use the PR

history, reproducible experiments and report together as evidence of engagement. *Commit counts and the presence of AI-generated code are not themselves grading criteria.* Do not fabricate results or retrieve hidden test labels.

Final submission checklist

☐ Final probabilities submitted to Kaggle for every `test.csv` ID.

☐ Completed report, with all required answers, tables, figures and sources, submitted as PDF.

☐ GitHub PR linked on the report title page and in the course submission portal.

☐ Code and dependencies recreate final predictions (within reasonable numerical tolerance).

☐ Relevant experiments and seed/configuration details are recorded.

☐ AI-use disclosure is complete; hidden labels were not retrieved externally.

Appendix A. Pretrained models in the submitted blends

All checkpoints come from the Hugging Face Hub. The from-scratch models (C1–C3, files D–F) use no pretrained weights.

Table 1: The pretrained checkpoints inside blends A–C, and how each is used.

Checkpoint (Hugging Face)	Params	How we used it	In blends
Frozen language models: token statistics as features for a LightGBM head (weights never updated)			
`openai-community/gpt2` [1]	124M	per-token log-prob, rank, entropy (15 stats)	A, B, C
`openai-community/gpt2-medium` [1]	355M	same statistics, larger LM	A, B, C
`Qwen/Qwen2.5-0.5B` + `-Instruct` [2]	0.5B	Binoculars observer/performer pair [3]: perplexity, cross-perplexity, Binoculars score	A, B, C
`Qwen/Qwen2.5-1.5B` + `-Instruct` [2]	1.5B	same, larger pair	A, B, C
Encoders fine-tuned end to end as classifiers (5-fold; test score = mean of the fold models)			
`FacebookAI/roberta-base` [4]	125M	fine-tuned classifier	A, C
`microsoft/deberta-v3-large` [5]	435M	fine-tuned classifier	A
`openai-community/roberta-large-openai-detector` [6]	355M	GPT-2 output detector, fine-tuned further	A

Used for evaluation only, not in any submitted model. The out-of-distribution proxy (Q5) contains AI text that we generated locally with seven open models: Llama-3.2-3B-Instruct [20], Qwen3-4B-Instruct-2507 [21], Phi-3.5-mini-instruct [22], SmolLM2-1.7B-Instruct [23], Granite-3.1-2B-Instruct [24], OLMo-2-1B-Instruct [25] and Pythia-1.4B [26]. That text was scored, never trained on.

Tried during development, not submitted. DeBERTa-v3-base/small [5], DistilRoBERTa [7], ModernBERT-base [8], ELECTRA-base [9], the RoBERTa-base GPT-2-output detector [6], and BGE [10] and E5 [11] sentence embeddings.

Appendix B. Kaggle submissions

Every file we uploaded, in upload order. E and F were first uploaded as v1 and then replaced by v2 under the same letter (Q5). Validation = the fixed 2,000 rows. The full testing history of every model we validated, including those never uploaded, is in `SUBMISSIONS.md` in the repository.

File	Model	Val. AUROC	Public	Private	Commit
A	Blend A (final): 7 models, proxy-chosen weights	0.99990	0.99161	–	`c7c1e3e`
B	Blend B: 4 frozen-LM heads, equal weights	0.99985	0.98848	–	`c7c1e3e`
C	Blend C: blend B + RoBERTa-base	0.99987	0.98845	–	`c7c1e3e`
D	C2 stylometric GBM	0.99831	0.95995	–	`08cde91`
E (v1)	C1 TF-IDF + LR, first version	0.96256	0.88583	–	`d0cfc4c`
F (v1)	C3 Transformer, first version	0.98500	0.83201	–	`a067917`
E	C1 TF-IDF + LR, v2 (raw char + distorted n-grams)	0.99678	0.96374	–	`05e0417`
F	C3 Transformer, v2 (lines joined, distorted)	0.98903	0.91502	–	`05e0417`

References

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. Language Models are Unsupervised Multitask Learners. OpenAI technical report, 2019.

[2] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

[3] A. Hans et al. Spotting LLMs With Binoculars: Zero-Shot Detection of Machine-Generated Text. ICML 2024; arXiv:2401.12070.

[4] Y. Liu et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019.

[5] P. He, J. Gao, W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. ICLR 2023; arXiv:2111.09543.

[6] I. Solaiman et al. Release Strategies and the Social Impacts of Language Models. arXiv:1908.09203, 2019 (the GPT-2 output detectors).

[7] V. Sanh, L. Debut, J. Chaumond, T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108, 2019 (DistilRoBERTa follows the same recipe).

[8] B. Warner et al. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference (ModernBERT). arXiv:2412.13663, 2024.

[9] K. Clark, M.-T. Luong, Q. V. Le, C. D. Manning. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. ICLR 2020.

[10] S. Xiao et al. C-Pack: Packed Resources For General Chinese Embeddings (BGE models). SIGIR 2024; arXiv:2309.07597.

[11] L. Wang et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training (E5). arXiv:2212.03533, 2022.

[12] T. Wolf et al. Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020 (System Demonstrations).

[13] G. Ke et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.

[14] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, 2011.

[15] Y. Ma et al. AI vs. Human – Differentiation Analysis of Scientific Content Generation. arXiv:2301.10416, 2023.

[16] E. Stamatatos. Authorship Attribution Using Text Distortion. EACL 2017.

[17] R. Sennrich, B. Haddow, A. Birch. Neural Machine Translation of Rare Words with Subword Units. ACL 2016.

[18] A. Vaswani et al. Attention Is All You Need. NeurIPS 2017.

[19] R. Caruana, A. Niculescu-Mizil, G. Crew, A. Ksikes. Ensemble Selection from Libraries of Models. ICML 2004.

[20] Llama Team, AI @ Meta. The Llama 3 Herd of Models. arXiv:2407.21783, 2024 (Llama 3.2 checkpoints).

[21] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025.

[22] M. Abdin et al. Phi-3 Technical Report. arXiv:2404.14219, 2024 (Phi-3.5 checkpoints).

[23] L. Ben Allal et al. SmolLM2: When Smol Goes Big. arXiv:2502.02737, 2025.

[24] IBM Granite Team. Granite 3.1 language models (model card, `ibm-granite/granite-3.1-2b-instruct`), 2024.

[25] Team OLMo. 2 OLMo 2 Furious. arXiv:2501.00656, 2024.

[26] S. Biderman et al. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. ICML 2023.

[27] Project Gutenberg. https://www.gutenberg.org.